TECHNISCHE UNIVERSITÄT
ILMENAU

Institut für Technische Informatik und Ingenieurinformatik
Fakultät für Informatik und Automatisierung
Fachgebiet System- und Software-Engineering

Bachelorarbeit

Modellgetriebene Microservice-Entwicklung

vorgelegt von

Patrik Löffler
Matrikelnummer 56207

Betreuer:

Prof. Armin Zimmermann
Dr. Ralph Maschotta, Tino Jungebloud

Ilmenau, den 9. Oktober 2023

Inhaltsverzeichnis

1 Abstract **1**

2 Einleitung **3**

2.1 Einführung in das Thema . . . 3

2.2 Zielstellung . . . 3

3 Grundlagen **5**

3.1 Domain-Driven Design . . . 5

3.1.1 Definition . . . 5

3.1.2 Bausteine des Model-Driven Design . . . 5

3.1.3 Flexibilität des Designs . . . 8

3.1.4 Strategic Design . . . 10

3.1.5 Large-Scale Structure . . . 12

3.2 Microservices . . . 12

3.2.1 Definition . . . 12

3.2.2 Architektur von Microservice-Systemen . . . 12

3.2.3 Kommunikation . . . 13

3.2.4 Architektur eines Microservices . . . 13

3.2.5 Betrieb und Continious Delivery . . . 14

3.3 Cloud-Infrastruktur . . . 14

3.3.1 Definition . . . 14

3.4 Modellgetriebene Entwicklung von Microservices . . . 14

3.4.1 AjiL - an Eclipse tool for MSA modeling with a code generator . . . 14

3.4.2 OCCIware - a unified framework for managing diverse cloud resources and services . . . 14

3.4.3 ARGON - automated cloud infrastructure provisioning using a domain-specific language . . . 15

3.4.4 model-driven approach for adaptive microservice-based systems using VxBPMN4MS . . . 16

3.4.5 Context Mapper, a tool for DDD and microservice design. . . . 16

3.4.6 DOML simplifies infrastructure deployments by unifying multiple IaC languages . . . 17

4 Modellierung **19**

1 Abstract

2 Einleitung

2.1 Einführung in das Thema

2.2 Zielstellung

Ziel der Arbeit ist es, Anwendungen, die das Microservice-Paradigma verwenden, mit Werkzeugen und Methoden der modellgetriebenen Softwareentwicklung zu entwickeln.

Dazu wird ein Metamodell konzipiert, welches die grundlegenden Eigenschaften des Paradigmas abstrahiert. Ergänzt wird dies durch die Berücksichtigung von Konzepten des Domain-Driven Design zur Modellierung der Geschäftslogik.

Die mithilfe dieses Metamodells generierten Anwendungen sollen in der Lage sein, bereits existierende, sogenannte Microservice Chassis – wiederverwendbare Grundbausteine eines Microservices – zu verwenden. Daraufhin wird erforscht, welche Möglichkeiten es gibt, diese generierten Anwendungen automatisiert, in Abhängigkeit von Versionsaktualisierungen der Chassis, zu migrieren.

Außerdem wird untersucht, inwiefern die der Anwendung zugrunde liegenden IT-Infrastruktur in solch einem Metamodell berücksichtigt werden kann und welche Möglichkeiten sich daraus ergeben.

3 Grundlagen

3.1 Domain-Driven Design

3.1.1 Definition

Modelle sind bewusst simplifizierte und strukturierte Darstellungen von Wissen. Die Methodik des Domain-Driven Design (DDD) stellt drei Regeln auf, welche bei der Wahl eines Modells zu berücksichtigen sind: Modell und Implementierung beeinflussen sich gegenseitig. Das Modell ist die Grundlage der im Entwicklungsprozess verwendeten Terminologie. Das Modell beinhaltet Wissen in einer nutzbaren, aufgearbeiteten Form. [Eva07, S. 3-4]

Fachlichkeiten aus einer Domäne werden mit einer Unbiquitous Language, einer dem Model-basieten Sprache welche Entwickler und Domänen-Experten eine gemeinsame Terminologie vorgibt, ausgedrückt. [Eva07, S. 25-27]

Im Domain-Driven Design (DDD) bezeichnet und gewährleistet das Model-Driven Design(MDD) die möglichst enge Abstimmung zwischen Software-Design und konzeptionellem Domänenmodell. Um eine hohe Übereinstimmung zwischen Modell und Design zu erreichen, ist es fast unerlässlich, innerhalb eines von Software-Werkzeugen unterstützten Modellierungsparadigmas zu arbeiten. Ein ideales Werkzeug sind objektorientierte Programmiersprachen, da Modellkonzepte sich recht gut in objektorientierte Konzepte übersetzen lassen. [Eva07, S. 49-51]

3.1.2 Bausteine des Model-Driven Design

Folgende abstrakten Bausteine existieren im MDD:

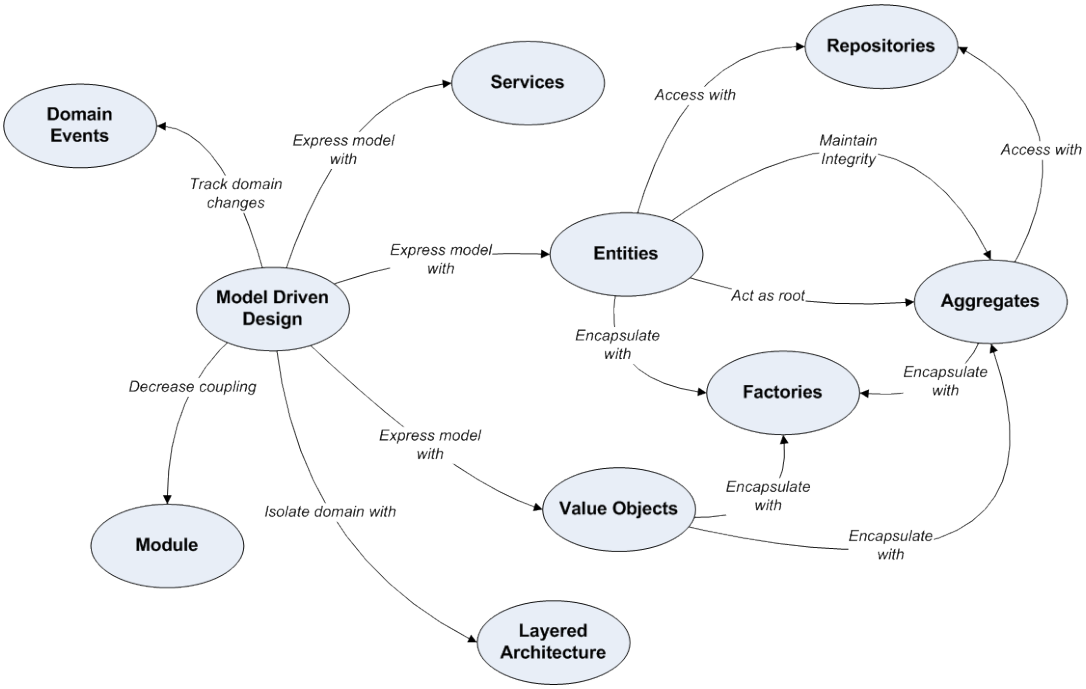

Abbildung 3.1: Bausteine des Model-Driven Design

[Eva07, S. 65]

Schichtenarchitektur

Um die Komplexität von Software zu beherrschen, werden unter anderem Schichtenarchitekturen verwendet. In dieser hängt jedes Element einer Schicht nur von anderen Elementen in der gleichen oder den darunterliegenden Schichten ab. Im Kontext des DDD ist hier die Trennung der Geschäftslogik in eine eigene Domain Layer entscheidend. Dies erzeugt eine starke Kohäsion zum Modell im Sinne des MDD. Mit Entwurfsmustern wie dem Model-View Controller kann auf diese Domain Layer wieder zugegriffen werden. [Eva07, S. 69 - 73]

Associations

Die Umsetzung von Assoziationen kann zwischen Modell und Implementierung stark variieren. Um die Assoziationen eines Modells besser handhabbar zu machen, gibt es verschiedene Ansätze: Das Festlegen einer Durchlaufrichtung verineinfacht beispielsweiße das Design und Reduziert Abhäbgigkeiten. Auch das Hinzufügen eines Qualifikators der effektiv die Multiplizität reduziert vereinfacht Modell und Implementierung. Das Entfernen von nicht essenziellen Assoziationen ist schließlich ebenso möglich. [Eva07, S. 81-85]

Entities

Ein Objekt, das hauptsächlich durch seine Identität definiert wird, wird als Entität bezeichnet. Dessen Attribute sind dabei nebensächlich. Eine Entität stellt eine abstrakte Kontinuität eines Objektes dar, die sich durch einen Lebenszyklus zieht. Klassendefinitionen, Verantwortlichkeiten, Attribute und Assoziationen sollten die Identität der Entität repräsentieren.

Bei einer Modellierung ist die grundlegendsten Verantwortlichkeit von Entitäten, Kontinuität herzustellen. Sie sollten sparsam verwendet werden. Nur das Verhalten das unerlässlich ist sollte ihnen angefügt werden. Darüber hinaus sollten zusätzliche Verhalten und Attribute zu anderen damit verbundenen Objekten verschoben werden. Die Identität soll mit einem einzigartigen Identifikationswert, welcher beispielsweiße automatisch generiert werden kann, sichergestellt werden. [Eva07, S. 89-96]

Value Objects

Ein Objekt, das einen beschreibenden Aspekt des Domänenbereichs darstellt und keine konzeptionelle Identität besitzt, wird als Wertobjekt (Value Object) bezeichnet. Wertobjekte können eine Zusammenstellung von anderen Objekten sein. Sie können Entitäten referenzieren. Wertobjekte werden häufig als Parameter in Nachrichten zwischen Objekten übergeben. Sie werden als Attribute von Entitäten (oder anderen Werten) verwendet. Die Felder eines Werteobjekts sollen ein konzeptionelles Ganzes formen.

Beim Design sollte das Verhalten beim kopieren, teilen und der Veränderlichkeit von Value Objekten berücksichtigt werden. Damit Value Objects sicher geteilt werden können müssen sie unveränderlich sein. Alternativ müssen Value Objekte als Kopien übergeben werden. Weiterhin sind bidirektionale Assoziationen zwischen Value Objekten aufgrund der fehlenden Identität bedeutungslos und sollten entfernt werden. [Eva07, S. 97-103]

Services

Einige Konzepte aus dem Domänenbereich sind nicht natürlich als Objekte zu modellieren. Daher wird ein neues Konzept eingeführt: Ein Service ist eine als Schnittstelle angebotene Operation, die im Modell eigenständig steht, ohne einen Zustand zu kapseln. Er wird rein nach dem definiert, was er für einen Client tun kann. Meist wird er nach einer Aktivität benannt und nicht nach einer Entität. Sie sollten bedacht eingesetzt werden und es sollte nicht zugelassen werden, dass sie den Entitäten und Wertobjekten all ihr Verhalten entziehen. Ein guter Service hat drei Eigenschaften: Die Operation bezieht sich auf ein Domänenkonzept, das kein natürlicher Teil einer Entität oder eines Wertobjekts ist. Die Schnittstelle ist in Bezug auf andere Elemente des Domänenmodells definiert. Die Operation ist zustandslos. Da Services auch außerhalb der Domain Layer auftreten können ist eine präzise fachliche Betrachtung nötig. [Eva07, S. 104 - 108]

Modules/Packages

Modules strukturieren Teile des Modells, sodass in ihnen eine möglichst hohe Kohäsion herrschst und zwischen ihnen eine möglichst lose Kopplung. Daher bündeln sie konzeptionell zusammengehörige Elemente. [Eva07, S. 109 - 116]

Aggregates

Ein Aggregat ist ein Cluster von assoziierten Objekten, die wir zu dem Zweck von Datenänderungen als eine Einheit behandeln. Jedes Aggregat hat eine Wurzel und eine Grenze. Die Grenze definiert, was sich innerhalb des Aggregats befindet. Die Wurzel ist eine einzelne, spezifische Entität innerhalb des Aggregats. Die Wurzel ist das einzige Mitglied des Aggregats, auf das Außenobjekte Referenzen halten dürfen, obwohl Objekte innerhalb der Grenze Referenzen untereinander halten dürfen. Entitäten, die nicht die Wurzel sind, haben eine lokale Identität, aber diese Identität muss nur innerhalb des Aggregats unterscheidbar sein, da kein Außenobjekt sie außerhalb des Kontexts der Wurzel-Entität sehen kann. [Eva07, S. 123 - 135]

Factories

Die Erstellung komplexer Objekte ist eine Verantwortung der Domänenschicht, dennoch gehört diese Aufgabe nicht zu den Objekten, die das Modell ausdrücken. Ein Programmelement, dessen Verantwortung die Erstellung anderer Objekte ist, wird als Fabrik (Factory) bezeichnet. Eine Fabrik kapselt das Wissen ein, das zur Erstellung eines komplexen Objekts oder Aggregats benötigt wird. Zwei grundlegende Anforderungen sind:

Jede Erstellungsmethode ist atomar und erzwingt alle Invarianten des erstellten Objekts oder Aggregats. Eine Fabrik sollte nur in der Lage sein, ein Objekt in einem konsistenten Zustand zu produzieren.

Die Fabrik sollte abstrakt auf den gewünschten Typ ausgerichtet sein, anstatt auf die konkrete(n) erstellte(n) Klasse(n).

Obwohl eine Fabrik dafür verantwortlich ist sicherzustellen, dass alle Invarianten erfüllt sind, kann sie die Überprüfung von Invarianten delegieren, was oft am besten ist. Eine Fabrik kann für die Erstellung oder Rekonstitution verwendet werden. Die zweite hat zwei wesentliche Unterschiede:

Eine Entitäten-Fabrik, die zur Rekonstitution verwendet wird, weist keine neue Tracking-ID zu.

Eine Fabrik, die ein Objekt rekonstituiert, wird Verstöße gegen eine Invariante anders behandeln, da ein reines Ignorieren möglicherweise nicht ausreicht. [Eva07, S. 136 - 146]

Repositories

Ein Repository stellt alle Objekte eines bestimmten Typs als ein konzeptuelles Set dar. Es verhält sich wie eine Sammlung, jedoch mit einer ausgefeilteren Abfragefähigkeit. Objekte des entsprechenden Typs werden hinzugefügt und entfernt, und die Maschinerie hinter dem Repository fügt sie in die Datenbank ein oder löscht sie daraus. Clients fordern Objekte aus Repositories an, indem sie Abfragemethoden verwenden, die Objekte basierend auf vom Client angegebenen Kriterien auswählen, wie zum Beispiel Werte bestimmter Attribute. Diese Abfragen können in ihrer Komplexität stark variieren. [Eva07, S. 147 - 161]

Specification

-Description [Eva07, S. 224 - 241]

3.1.3 Flexibilität des Designs

Verschiedene Muster existieren um im MDD Flexibilität zu ermöglichen:

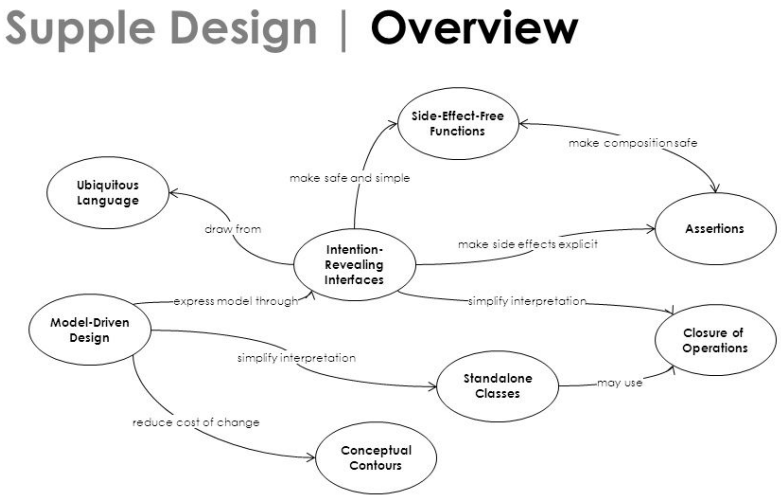

Abbildung 3.2: Supple Design

[Eva07, S. 245]

Intention-Revealing Interfaces

-Description [Eva07, S. 246 - 249]

Side Effect Free Functions

-Description [Eva07, S. 250 - 254]

Assertions

-Description [Eva07, S. 255 - 259]

Conceptual Contours

-Description [Eva07, S. 260 - 264]

Standalone Classes

-Description [Eva07, S. 265 - 267]

Closure of Operations

-Description [Eva07, S. 268 - 270]

Declarative Design

-Description [Eva07, S. 270 - 282]

Angles of Attack

-Description [Eva07, S. 282 - 292]

3.1.4 Strategic Design

Strategien im MDD:

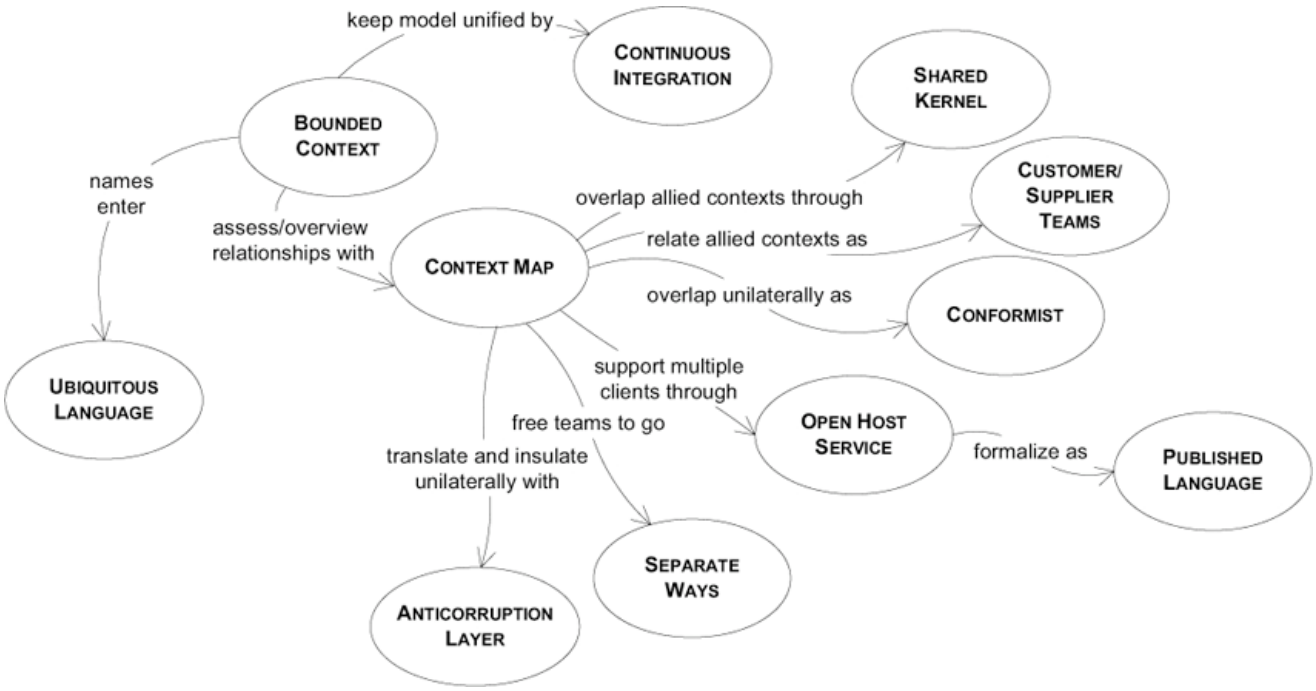

Abbildung 3.3: Supple Design

[Eva07, S. 334]

Bounded Context

-Description [Eva07, S. 335 - 340]

Continous Integration

-Description [Eva07, S. 341 - 343]

Context Map

-Description [Eva07, S. 344 - 353]

Shared Kernel

-Description [Eva07, S. 354 - 355]

Customer/Supplier

-Description [Eva07, S. 356 - 360]

Conformist

-Description [Eva07, S. 361 - 363]

Anticorruption Layer

-Description [Eva07, S. 364 - 370]

Seperate Ways

-Description [Eva07, S. 371 - 373]

Open Host Service

-Description [Eva07, S. 374]

Published Language

-Description [Eva07, S. 375-378]

Model Context Strategy

-Description [Eva07, S. 381-389]

3.1.5 Large-Scale Structure

Evolving Order

-Description [Eva07, S. 444-446]

System Methapor

-Description [Eva07, S. 447-449]

Responsibility Layer

-Description [Eva07, S. 450-464]

3.2 Microservices

3.2.1 Definition

Der zur Modularisierung eingesetzte Ansatz der Microservices erfüllt verschiedene Grundlegende Vorraussetzungen: In ihrer Implementierung gibt es keine technologischen Einschränkungen. Die Dienste können unabhängig voneinander überarbeitet und auf ein Zielsystem ausgerollt werden. Weiterhin sollen Build und Deployement automatisiert sein. Sie besitzen einen eigenen Datenhaushalt sowie eigenständige Prozesse auf der ausführenden Infrastruktureinheit. Microservices nutzen verteilte Netzwerkkommunikation, um sich mit anderen Diensten zu verständigen. Transaktionen zeichnen sich durch die ACID-Eigenschaften aus, zu denen Atomizität, Konsistenz, Isolation und Dauerhaftigkeit gehören. Es ist wichtig, dass Daten konsistent bleiben. Deshalb sollten keine Transaktionen über mehrere Services hinweg durchgeführt werden. Um diese Konsistenz zu gewährleisten, können Kompensationstransaktionen eingesetzt werden. [Wol16, S. 2] [Wol16, S. 31-37]

3.2.2 Architektur von Microservice-Systemen

Jeder Microservice soll eine eigene fachliche Einheit bilden, die so geschnitten ist, dass für Änderungen oder neue Features nur ein Microservice geändert werden muss. Weiterhin sollen die aus dem DDD bekannten Konzepte der Unbiquitous Language und des Bounded Context verwendet werden.

-DDD: Bounded Context: Jedes Domänenmodell ist nur in bestimmten Grenzen sinnvoll. (Stategic Design) Verschiedene Arten von Kollaboration zwischen Bounded Contexts: Shared Kernel, Customer/Supplier, Conformists, Anti Corruption Layer, Seperate Ways, Open Host Service, Published Language. Es sollte vermieden werden, dass Microservices mehrere Bounded Contexts enthalten.

- Large-Scale Strukturen: System Methapor, Responsibility Layer, Evolving Order. [Wol16, S. 44-53]

- ???: Jeder Microservice benötigt eine eigene Continous-Delivery-Pipeline. Jeder muss außerdem ein Monitoring haben. Unabhängige Versionskontrolle [Wol16, S. 80-81]

- Fachichliche Abhängigkeiten (Änderungen am Datenmdell z.B.):Lose Kopplung,Hohe Kohäsion,Zyklische Abhängigkeiten vermeiden [Wol16, S. 101-105]

- Context Maps zu Visualisierung von Bounded Contexts [Wol16, S. 109-111]

- Änderung der Gesamtarchitektur: Gemeinsame Bibliothek,Code übertragen, Gemeinsamer Service, Neuer Service, Neu schreiben [Wol16, S. 112-120]

- Microservice-Chassis

- Event-Driven Architecture; Emmiter/Consumer, lose Kopplung, impl durch Messaging [Wol16,

S. 137 - 138]

- Microservice Konfiguration [Wol16, S. 141 - 142]
- Service Discovery [Wol16, S. 143 - 146]
- Load Balancing [Wol16, S. 147 - 150]
- Sicherheit [Wol16, S. 153 - 161]

3.2.3 Kommunikation

- REST [Wol16, S. 179-182]
- SOAP [Wol16, S. 182 - 183]
- Messaging [Wol16, S. 183 - 187]
- Schnittstellen [Wol16, S. 190 - 193]

3.2.4 Architektur eines Microservices

- Fachliche Architektur [Wol16, S. 197 - 198]
- CQRS [Wol16, S. 199 - 201]
- Event Sourcing [Wol16, S. 201 - 203]
- Hexagonale Architektur [Wol16, S. 203 - 206]
- Resilience und Stabilität [Wol16, S. 207 - 212]

3.2.5 Betrieb und Continious Delivery

- Logging [Wol16, S. 244 - 249]
- Monitoring [Wol16, S. 250 - 256]
- Deployement [Wol16, S. 256 - 260]
- Infrastruktur [Wol16, S. 263 - 269]

3.3 Cloud-Infrastruktur

3.3.1 Definition

- Cloud, Virtualisierung, Continous Delivery, Container

[Ste20, S. 20] [Ste20, S. 46-63]

- Docker [Ste20, S. 131-166]

- Kubernetes [Ste20, S. 166-192]

3.4 Modellgetriebene Entwicklung von Microservices

3.4.1 AjiL - an Eclipse tool for MSA modeling with a code generator

AjiL is an Eclipse-based tool for graphical MSA modeling. It features a diagram editor for modeling rudimentary domain models, functional and infrastructure microservices, and provided and required interfaces. Thus, AjiL facilitates creating intermediate models for service interfaces (see the section “Intermediate Models”). AjiL ships with a code generator that produces microservice implementations from AjiL models. The generated Java code is based on Spring Boot and Spring Cloud. It is directly runnable and comprises interface implementations, stubs for implementing services’ business logic, and configuration files for infrastructure components. However, AjiL currently does not support round-trip engineering.4 The tool is available at github.com/SeelabFhdo/AjiL.

https://ieeexplore.ieee.org/abstract/document/8354426 https://github.com/SeelabFhdo/AjiL

3.4.2 OCCIware - a unified framework for managing diverse cloud resources and services

Cloud computing has emerged as the main paradigm for hosting and delivering computing resources as services over Internet. It provides a delivery model for computing resources at infrastructure, platform, and software levels. However, there is a plethora of cloud providers offering different resource management interfaces. Due to this diversity, the consumption, provisioning, management, and supervision of cloud resources are subjected to four key issues, i.e., heterogeneity, interoperability, integration, and portability. To cope with these issues, Open Cloud Computing Interface (OCCI) is proposed as a community-based and open recommendation standard for managing any kind of cloud resources. Currently, only runtime implementations exist for OCCI, and each one targets a specific cloud service model such as Infrastructure as a Service (IaaS), Platform as a Service (PaaS), or Software as a Service (SaaS). Thus, OCCI lacks an approach to model and execute different OCCI artifacts. Our approach provides a generic modeling framework coupled with a generic runtime implementation. In this article, we propose the OCCIware approach, which represents the first approach to design, validate, generate, implement, deploy, execute, and supervise everything as a service with OCCI. This approach provides OCCIware Studio, the first model-driven tool chain for OCCI. It is built around OCCIware Metamodel, which defines the static semantics for the OCCI standard in Ecore and OCL. In addition, it proposes OCCIware Runtime, the first generic OCCI runtime implementation targeting all the cloud service models (IaaS, PaaS, and SaaS). OCCIware provides a unique

and unified framework to manage OCCI artifacts and, at the same time, it represents a factory to build cloud domain-specific modeling frameworks where each framework targets a specific cloud domain. OCCIware has been applied in various cloud domains and use-cases, which validate its applicability.

https://www.sciencedirect.com/science/article/pii/S0167739X18306071# sec5

3.4.3 ARGON - automated cloud infrastructure provisioning using a domain-specific language

Infrastructure as Code (IaC) is an approach to infrastructure automation based on practices from software development. IaC tools use scripts to specify the creation, update, and execution of cloud infrastructures. As each cloud provider offers a different type of infrastructure, the definition of an infrastructure element implies to write several lines of code that strongly depend on the target cloud provider. As a result, managing IaC scripts has become a time-consuming and error-prone activity. In previous work, we have presented ARGON, which is a model-driven infrastructure provisioning tool. ARGON implements a domain-specific language (DSL) for modeling the characteristics of the cloud infrastructure and provides transformation engines to automate the infrastructure provisioning for different cloud providers. In this demonstration, we show the use of ARGON for modeling and provisioning a load balancer in Amazon Web Services and Microsoft Azure. The load balancer distributes incoming application traffic across multiple virtual machines.

https://ieeexplore.ieee.org/abstract/document/8904494

3.4.4 model-driven approach for adaptive microservice-based systems using VxBPMN4MS

A microservice-based system is composed of numerous independently deployed and executed microservices, among which normally exist the complex dependencies. Traditional service composition approaches usually expect the business process predefined at design time. As a result, it is difficult for the microservice-based system to quickly adapt to the frequently changing operation environments and business requirements. To address the above limitations, we propose a variability-enabling and model-driven approach to developing adaptive microservice-based systems. Our approach first models the business process with variability using VxBPMN4MS, an extension of Business Process Model and Notation (BPMN) with support for variability, then transforms the business process model to variability supported microservice composition frameworks, and finally derives business process instances at run-time according to the predefined process configuration. We have developed a platform to automate the proposed approach as much as possible, and conducted a case study to evaluate the effectiveness of the proposed approach and platform.

https://ieeexplore.ieee.org/abstract/document/9529482

3.4.5 Context Mapper, a tool for DDD and microservice design.

Service-oriented architectures and microservices have gained much attention in recent years; companies adopt these concepts and supporting technologies in order to increase agility, scalability, and maintainability of their systems. Decomposing an application into multiple independently deployable, appropriately sized services and then integrating such services is challenging. With strategic patterns such as Bounded Context and Context Map, Domain-driven Design (DDD) can support business analysts, (enterprise) architects, and microservice adopters. However, existing architecture description languages do not support the strategic DDD patterns sufficiently; modeling tools for DDD primarily focus on its tactical patterns. As a consequence, different opinions on how to apply strategic DDD exist, and it is not clear how to combine its patterns. Aiming for a clear and concise interpretation of the patterns and their combinations, this paper distills a meta-model of selected strategic DDD patterns from the literature. It then introduces Context Mapper, an open source project that a) defines a Domain-specific Language (DSL) expressing the strategic DDD patterns and b) provides editing, validation, and transformation tools for this DSL. As a machine-readable description of DDD, the DSL provides a modeling foundation for (micro-)service design and integration. The models can be refactored and transformed within an envisioned tool chain supporting the continuous specification and evolution of Context Maps. Our validation activities (prototyping, action research, and case studies) suggest that the DDD pattern clarification in our meta-model and the Context Mapper tool indeed can benefit the target audience.

https://www.scitepress.org/Papers/2020/89105/89105.pdf

3.4.6 DOML simplifies infrastructure deployments by unifying multiple IaC languages

One of the main DevOps practices is the automation of resource provisioning and deployment of complex software. This automation is enabled by the explicit definition of Infrastructure-as-Code (IaC), i.e., a set of scripts, often written in different modelling languages, which defines the infrastructure and applications to be deployed.

We introduce the DevOps Modelling Language (DOML), a new Cloud modelling language for infrastructure deployments. DOML is a modelling approach that can be mapped into multiple IaC languages, addressing infrastructure provisioning, application deployment and configuration at once. The idea behind DOML is to use a single modelling paradigm which can help to reduce the need of deep technical expertise in using different specialised IaC languages.

We present the DOML's principles and discuss the related work on IaC languages. We demonstrate the DOML advantages for the end-user in comparison with state-of-the-art IaC languages

such as Ansible, Terraform, and Cloudify, and show its effectiveness through an example.

4 Modellierung

Literaturverzeichnis

[Eva07] Eric Evans. *Domain-driven design: tackling complexity in the heart of software*. Addison-Wesley, Upper Saddle River, NJ [u.a.], 11. printing edition, 2007. Eric Evans.

[Ste20] Daniel Stender. *Cloud-Infrastrukturen: das Handbuch für DevOps-Teams und Administratoren*. Rheinwerk computing. Rheinwerk Computing, Bonn, 1. auflage edition, 2020. Daniel Stender.

[Wol16] Eberhard Wolff. *Microservices: Grundlagen flexibler Softwarearchitekturen*. dpunkt.verlag, Heidelberg, 1., korrigierter nachdruck edition, 2016. Eberhard Wolff.